

PROGRAMMING IN JAVA

Evaluated Project Report

PhysioClinic Appointment Queue System

Command-Line Physiotherapy Clinic Scheduling and Priority Queue Management

Student Name	[ENTER YOUR NAME]
Registration Number	[ENTER REGISTRATION NUMBER]
Programme / Branch	[ENTER PROGRAMME / BRANCH]
Course	Programming in Java
Institution	VIT Bhopal University
Submission Platform	VITyarthi
Project Type	Build Your Own Project
Date	September 18, 2026

This report is structured to match the VITyarthi Build Your Own Project requirements. Student-specific details and final submission screenshots should be customized before portal submission.

1. Introduction

PhysioClinic Appointment Queue System is a Java command-line application for a small physiotherapy clinic. It organizes patient registration, therapist working hours, appointment booking, priority-based queue processing, appointment lifecycle management, and daily reporting. The project demonstrates Java object-oriented programming, collections, file persistence, validation, exception handling, and modular software design.

The solution deliberately avoids external runtime dependencies. An evaluator with Java 17 or later can clone the public GitHub repository, compile the source files from a terminal, run the application, and execute the included test runner.

2. Problem Statement

Small clinics can face duplicated appointment slots, unclear therapist availability, and inconsistent handling of urgent cases when scheduling is done manually. A lightweight scheduling tool can centralize these tasks, validate booking rules, and use a priority queue to ensure that emergency appointments are considered before routine appointments.

The project scope is limited to administrative scheduling. It does not provide medical diagnosis, treatment advice, billing, insurance processing, or production healthcare compliance.

3. Functional Requirements

ID	Requirement
FR-01	Register patients with unique IDs and validated information.
FR-02	Search patient records by name.
FR-03	Register therapists with specialization and working hours.
FR-04	Book appointments with date, time, priority, and notes.
FR-05	Reject appointment times outside therapist working hours.
FR-06	Prevent therapist double-booking.
FR-07	Prevent the same patient from conflicting appointments.
FR-08	Prioritize waiting appointments using Java PriorityQueue.
FR-09	Call the next appointment and set it to IN_PROGRESS.
FR-10	Complete or cancel appointments according to status rules.
FR-11	Generate daily status summaries.
FR-12	Persist project data to local text files.

4. Non-Functional Requirements

ID	Requirement
NFR-01	Reliability: conflict checks and status validation prevent invalid scheduling states.
NFR-02	Usability: numbered CLI menu, clear prompts, and controlled error messages.
NFR-03	Maintainability: separate packages for models, repositories, services, UI, utilities, and tests.

ID	Requirement
NFR-04	Performance: PriorityQueue provides efficient priority insert/remove operations for the queue.
NFR-05	Privacy: phone numbers are masked in user-facing displays and full patient contact details are not logged.
NFR-06	Portability: standard Java 17+ APIs only; no GUI or database server required.
NFR-07	Error handling: validation and exceptions are converted into readable CLI messages.
NFR-08	Resource efficiency: local text files and in-memory Java collections are used for this academic scope.

5. System Architecture

The application uses a lightweight layered architecture. ConsoleUI handles interaction, ClinicService implements business rules, repositories handle persistence, model classes represent entities, and utilities provide validation, privacy masking, ID generation, and logging.

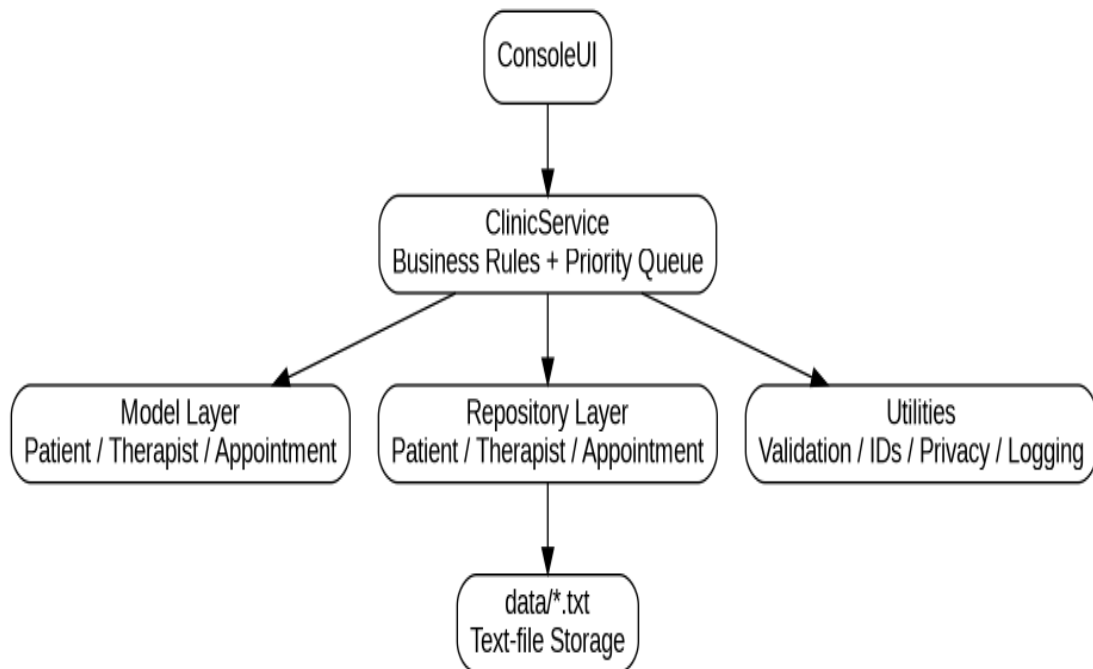


Figure 1. Layered system architecture.

6. Design Diagrams

6.1 Use Case Diagram



Figure 2. Use cases for clinic staff.

6.2 Workflow Diagram

Start -> Main Menu -> Register / Therapist / Book Appointment -> Validate -> Save as WAITING -> Priority Queue -> Call Next -> IN_PROGRESS -> Complete or Cancel -> Report / Exit

Booking validation branches reject missing records, invalid times, and schedule conflicts before persistence.

6.3 Sequence Diagram

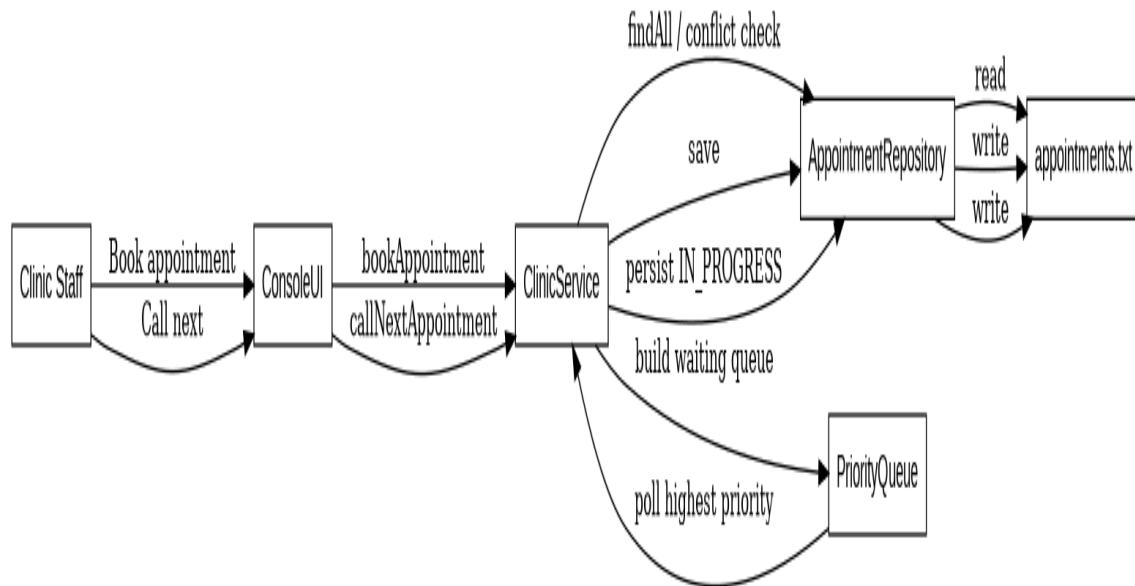


Figure 3. Appointment booking and queue-processing interaction.

6.4 Class / Component Diagram

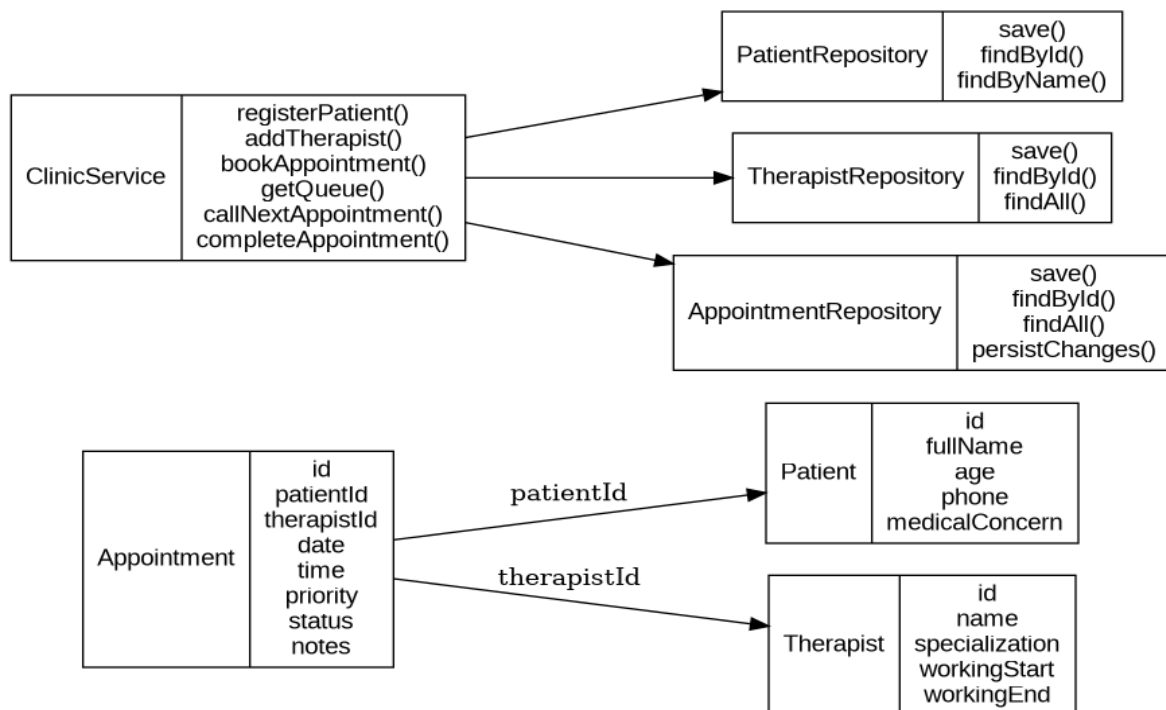


Figure 4. Core classes and repository relationships.

6.5 Storage / ER View

The project uses file-based persistence instead of an SQL database. One patient can have many appointments, and one therapist can have many appointments.

```

PATIENT (patient_id, name, age, phone, concern) 1 --- N APPOINTMENT (appointment_id, patient_id, therapist_id,
date, time, priority, status, notes) N --- 1 THERAPIST (therapist_id, name, specialization, start_time,
end_time)
  
```

7. Design Decisions & Rationale

Decision	Rationale
PriorityQueue	Maps naturally to emergency/urgent/routine service order. Appointment.compareTo() keeps queue
Layered packages	Separates business logic, persistence, interface, and utilities for easier maintenance.
File persistence	Keeps the project self-contained and runnable with a standard JDK while demonstrating persistence
Explicit status lifecycle	WAITING, IN_PROGRESS, COMPLETED, and CANCELLED statuses provide controlled state transition
Dependency-free tests	An executable test runner avoids external dependency download during evaluation.
Privacy-aware display	Phone numbers are masked and queue views use patient initials, reducing exposure of contact d

8. Implementation Details

The implementation contains more than the minimum five to ten meaningful Java modules/classes. The main structure is shown below.

Package / File	Responsibility
model.Patient	Stores patient details and registration timestamp.
model.Therapist	Stores specialization and working hours.
model.Appointment	Stores appointment data and implements Comparable for queue ordering.
model.PriorityLevel	Defines EMERGENCY, URGENT, ROUTINE ranks.
model.AppointmentStatus	Defines appointment lifecycle states.
service.ClinicService	Business rules, validation, queue operations, reports.
ui.ConsoleUI	Command-line menu and user interaction.
repository.*	File-based persistence for patients, therapists, appointments.
util.*	Validation, privacy masking, logging, ID generation.
tests.TestRunner	Executable dependency-free automated tests.

The source uses Java 17 language/API compatibility and can be compiled with javac. No graphical interface or external server is required.

9. Screenshots / Results

A demonstration run was executed with a sample therapist, a sample patient, one routine appointment, and one emergency appointment. The emergency appointment was returned first by the PriorityQueue. The included test runner completed six validation tests successfully.

```
PhysioClinic - Demonstration Result

$ bash test.sh
[PASS] Patient registration
[PASS] Successful appointment booking
[PASS] Double-booking prevention
[PASS] Priority queue ordering
[PASS] Call next and complete
[PASS] Working-hours validation
Passed: 6    Failed: 0

Priority queue demonstration:
Position  Appt ID   Patient  Therapist  Priority
1         A0002    RD       T001       EMERGENCY
2         A0001    RD       T001       ROUTINE

Call next -> A0002 -> IN_PROGRESS -> COMPLETED

Daily report for 2026-09-18:
Total: 2    Waiting: 1    In Progress: 0
Completed: 1    Cancelled: 0
```

Figure 5. Demonstration output showing automated tests, priority queue order, status processing, and daily report.

10. Testing Approach

Test Case	Expected Result
Patient registration	PASS - unique patient ID generated and record stored.
Successful appointment booking	PASS - valid appointment stored as WAITING.
Double-booking prevention	PASS - same therapist/date/time rejected.
Priority queue ordering	PASS - EMERGENCY appears before ROUTINE.
Call and complete	PASS - WAITING becomes IN_PROGRESS then COMPLETED.
Working-hour validation	PASS - out-of-hours booking rejected.

Command used: `bash test.sh`. The current verified result is 6 passed and 0 failed.

11. Challenges Faced

- Preventing double-booking requires checking existing active appointments before saving a new appointment.
- Priority handling needs a deterministic comparator so equal-priority appointments are ordered consistently.
- Persisted text files can contain malformed records, so repository loading ignores invalid lines rather than making the application fail at startup.
- Console interaction needs validation and controlled exception handling because all user input arrives as text.
- Privacy-aware output must balance operational usability with reduced exposure of patient contact details.

12. Learnings & Key Takeaways

- Applied encapsulation, enums, constructors, methods, object relationships, and interfaces through Java OOP.
- Used PriorityQueue and Comparable to implement a practical priority scheduling algorithm.
- Learned how repositories can isolate persistence from business logic.
- Implemented defensive validation and domain-specific error messages.
- Used file I/O and `java.util.logging` to provide persistence and traceability without external services.
- Designed a project that can be compiled and tested entirely from the command line.

13. Future Enhancements

- Role-based authentication for reception staff and therapists.
- Encrypted database storage and stronger healthcare privacy controls for real deployment.
- Calendar views and appointment rescheduling.
- Wait-time estimation using queue statistics.
- Exportable reports in CSV/PDF format.
- Unit testing with JUnit and integration testing around persistence.
- Optional REST API and web/mobile front end in a future version.

14. References

- VITyarthi - Build Your Own Project: General Project Instructions & Submission Guidelines (course-provided document).
- Java Platform Standard Edition API documentation, especially `java.util.PriorityQueue`, `java.time`, `java.nio.file`, and `java.util.logging`.
- Project source code and diagrams contained in the accompanying GitHub repository.

15. Submission Checklist

Item	Status / Action
Public GitHub repository	Create public repository and submit root URL only.

Item	Status / Action
README.md	Included; verify clone/run instructions before submission.
statement.md	Included.
Source code	Included and modular.
Diagrams	Included as Markdown/Mermaid and Graphviz files plus report figures.
Testing	Executable test runner included; verify final run before submission.
Project report PDF	This file; replace student placeholders and add final personal evidence.
Original work	Customize code/report and maintain your own Git history before submission.